SEQUENCES

CS1910

Computer Science I

CODE EXAMPLES

- https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/09-Sequences/CS1910_Sequences_Workbook.ipynb

SEQUENCE

- Compound data type consisting of one of more pieces of data in a specific order
- An example sequence is a string

```
my_string = "HELLO"
```



Multiple pieces of data in a specific order

INDEXING

- Each data item in a sequence has a position (aka an offset)
- The position is denoted using an integer

Character	H	E	L	L	O
Offset Position	0	1	2	3	4

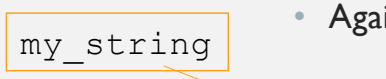
- This string has 5 characters, the first character is at offset 0 the last character is at offset 4
  - Starting our counts from 0 takes some practice / getting used to

INDEXES

- To access a character from a string you can use square brackets [ ]
- Example:

```
my_string[1]
# accesses my_string at offset 1 (or the letter "E")
```

- Although we say this accesses a character or letter from the string, what is returned is also a string (just a string with one character)
- Again, note that offset 1 is the 2nd character position or E (not H)



Character	H	E	L	L	O
Offset Position	0	1	2	3	4

INDEXES

- Offsets can be negative
  - -1 is the last letter in a string it is preceded by
  - -2 is 2nd last letter in a string and so on
- `IndexError: string index out of range`
  - Error produced if you try to access an index position that is off the end of the string
  - Example:

```
my_string = "Hello"
#two examples of IndexErrors
print(my_string[-6])
print(my_string[5])
```

- Indexes can also be expressions:

```
i = 0
print(my_string[i+2])
```

Must be an integer

```
#quick hitters

my_string = "Be a friend, pick me up off the ground"

print(my_string[2])
print(my_string[-2])
print((my_string[1]+my_string[-1])[1])
print(my_string[1][0][0][0][0])
```

QUICKHITTERS

```
#quick hitters

my_string = "Be a friend, pick me up off the ground"

print(my_string[2])
print(my_string[-2])
print((my_string[1]+my_string[-1])[1])
print(my_string[1][0][0][0][0])
```

```
n
d
e
```

QUICKHITTERS

Single letter string is still a string so [ ] index can be apply to it, and to that and, …

SELF-CHECK

What is the output?

```
my_string = "sdrawkcab"
for i in range(1, 10):
    print(my_string[-i])
```

SELF-CHECK

What is the output?

```
my_string = "sdrawkcab"
for i in range(1, 10):
    print(my_string[-i])
```

backwards

SLICING

- Selecting zero of more items of a sequence and forming them into a new sequence
- Uses the slicing operator **:**
- Slices are inclusive of the lower bound but exclusive of the upper bound
- Include a step size (optional)

```
slice = my_string[start:end:step]
```

Returns characters between start up to but not including end taking a step size of step

SLICING STEP SIZE

- A second colon and 3rd integer specifies a step size
- Allows every 2nd or 3rd or other .. Character to be selected

```
#Slicing Examples
    s = "Blueberry"

    # Every other character from the whole string
    # start at index 0, go to len(s), step by 2
    print(s[0:len(s):2])   # Bubry'

    # Every third character between offsets 2 (inclusive) and 7 (exclusive)
    print(s[2:7:3])       # 'ue'
```

SLICING DEFAULTS

- General form: seq[start:stop:step]
- Defaults:
  - start defaults to 0 (or len(seq)-1 if step is negative)
  - stop defaults to len(seq) (or -1 conceptually for negative steps)
  - step defaults to 1stop is exclusive.
  - Negative step walks backwards (handy for reversals: seq[::-1]).

```
t = "Charlottetown"

    # Reverse the whole string
    print(t[::-1])       # 'nwotte tl o tralhC' (no spaces in the
    actual output: 'nwottetolrahC')

    # Every second character in reverse, starting from the last
    character down to index 3 (exclusive)
    print(t[:3:-2])      # takes indices len(t)-1, len(t)-3, ...
    down to > 3
```

SLICING EXAMPLES

```
my_string = "Be a friend, pick me up off the ground"

print(my_string[0:1]) #first letter
print(my_string[:1]) #first letter
print(my_string[:11]) #first 11 letters (up to but not including the 11th offset)
print(my_string[-1:]) #last letter
print(my_string[-2:]) #last 2 letters
print(my_string[37:]) #last letter
print(my_string[37:len(my_string)]) #last letter
print(my_string[::2]) #every second letter
print(my_string[::3]) #every 3rd letter
print(my_string[6:12:2]) #every second letter in a slice
```

FIND A PARTNER

- In one-on-one basketball games, there are often the following rules:
  - Games go to 11 but you must win by 2
    - If the game is tied 10-10 then the first person to gain an advantage of 2 points is the winner
  - Each shot is either worth 1 or 2 points
  - Consider that the record of a one-on-on basketball game are given as a single string with the order of the points scored. The form is the player (A or B) who scored the points followed by an integer giving the number of points they scored.
- For example:
  - A2A2A2A1A2A1B2B2B2B2B2B2
- The above gives a game in which player A scores the first 10 points but then player B scores the next 12 thus winning the game (first to 11 (win by 2))

FIND A PARTNER

- Write a program that given a record of a one-on-one basketball game prints out the winner of the game
- Start with Computational Thinking
  - Decompose the problem
  - Identify Patterns
  - Abstraction
    - Filter out the unnecessary details
  - Validation
    - How can you tell if your program is working correctly

TRAVERSAL

- Traversing (or systematically visiting) each character in a string can be done with a for loop

```
# example traversals

my_string = "hello, world!"

# for in sequence loop
for char in my_string:
    print(char)

# for in range( ) using [ ] index
for i in range(len(my_string)): # note the use of len(my_string)
    print(my_string[i])

#while loop with [ ] index accessor
count = 0
while count < len(my_string):
    print(my_string[count])
    count += 1
```

What happens if we forget the `count += 1` line ?

SELF-CHECK

Which of these traversals will print out every second letter in my_string

```
my_string = "1234567890"
```

1

```
for ch in my_string[::2]:
    print(ch)
```

2

```
for i in range(0,len(my_string),2):
    print(my_string[i])
```

3

```
count = 0
while count < len(my_string):
    print(my_string[count])
    count += 2
```

SELF-CHECK

Which of these traversals will print out every second letter in my_string

```
my_string = "1234567890"

for ch in my_string[::2]:
    print(ch)

for i in range(0,len(my_string),2):
    print(my_string[i])

count = 0
while count < len(my_string):
    print(my_string[count])
    count += 2
```

They all print every second letter

STRINGS ARE IMMUTABLE

```
#String immutability - the [] cannot appear on the left side of an assignment
greeting = "Hello, world!"
# Attempt to replace the first character by assigning into an index
greeting[0] = "Y“
```

```
TypeError     Traceback (most recent call last)

Cell In[1], line 4

     2 greeting = "Hello, world!"

     3 # Attempt to replace the first character by assigning into an index

----> 4 greeting[0] = "Y"

TypeError: 'str' object does not support item assignment
```

- You can create a new string with new contents but you cannot change the letters inside an existing string

```
#String immutability - the [] cannot appear on the left side of an assignment
greeting = "Hello, world!"
# Attempt to replace the first character by assigning into an index
greeting = “Jello world“
```

- Printing greeting[0] would now print a ‘J’ as opposed to an ‘H’

A sample of useful string methods

STRING METHODS

- See code examples
  - `upper() return a copy of the string that is converted to UPPER CASE`
  - `find(other) return the first index position of a given other string`
  - `in` operator returns **True** if a given string is found and **False** otherwise
  - `strip()` returns a string with white space removed from start and end
  - `replace(old, new)` returns a new string with all occurrences of old replaced by new

STRING COMPARISON

- Recall from previous classes that comparison of strings is based on the ordinal value associated with the characters. Capital letters come before lower case.

```
word = input("Enter a word related to PEI: ")

if word < "Charlottetown":
    print("Your word, " + word + ", comes before Charlottetown.")
elif word > "Charlottetown":
    print("Your word, " + word + ", comes after Charlottetown.")
else:
    print("A perfect match—Charlottetown!")
```

SELF-CHECK

```
#self check what does this output?

count_str = "01234567890123456789012345678901234567"
my_string = "it's raining, get under my bumbershoot"
my_string.replace("bumbershoot","umbrella")
if "umb" in my_string[18:31:6]:
    print(my_string)
```

Remember strings are immutable

TRY AT HOME

- Write a program that prints all of the letters that are in common between two strings
- Write a program that beats the "Simon says game"
- Print all of the letters from a string in reverse order – use a while loop – use a for sequence loop
  - Each letter on their own line